

Université Sorbonne Paris Nord

École d'Ingénieurs Sup Galilée

Base de Données Avancées
TP 2

Ahmed SEHILI

2024/2025

Exercise 1

```
-- 1
SELECT dept_name
FROM department
WHERE budget = (SELECT MAX(budget) FROM department);

-- 2
SELECT name, salary
FROM teacher
WHERE salary > (SELECT AVG(salary) FROM teacher);

-- 3
SELECT th.ID, s.ID, COUNT(*)
FROM takes t
JOIN teaches th ON t.course_id = th.course_id
                AND t.sec_id = th.sec_id
                AND t.semester = th.semester
                AND t.year = th.year
JOIN student s ON t.ID = s.ID
GROUP BY th.ID, s.ID
HAVING COUNT(*) > 2;

-- 4
SELECT th.ID, s.ID, COUNT(*)
FROM takes t
JOIN teaches th ON t.course_id = th.course_id
                AND t.sec_id = th.sec_id
                AND t.semester = th.semester
                AND t.year = th.year
JOIN student s ON t.ID = s.ID
GROUP BY th.ID, s.ID
HAVING COUNT(*) > 2;

-- 5
SELECT ID, name
FROM student
WHERE ID NOT IN (
    SELECT DISTINCT ID
    FROM takes
    WHERE year < 2010
);

-- 6
SELECT name
FROM teacher
WHERE name LIKE 'E%';

-- 7
SELECT name, salary
FROM (
```

```

        SELECT name, salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
        FROM teacher
    )
    WHERE rnk = 4;

-- 8
SELECT name, salary
FROM (
    SELECT name, salary
    FROM teacher
    ORDER BY salary ASC
)
WHERE ROWNUM <= 3
ORDER BY salary DESC;

-- 9
SELECT name
FROM student
WHERE ID IN (
    SELECT ID
    FROM takes
    WHERE semester = 'Fall' AND year = 2009
);

-- 10
SELECT name
FROM student
WHERE ID = SOME (
    SELECT ID
    FROM takes
    WHERE semester = 'Fall' AND year = 2009
);

-- 11
SELECT name
FROM student NATURAL JOIN takes
WHERE semester = 'Fall' AND year = 2009;

-- 12
SELECT name
FROM student s
WHERE EXISTS (
    SELECT 1
    FROM takes t
    WHERE t.ID = s.ID
    AND semester = 'Fall'
    AND year = 2009
);

-- 13
SELECT DISTINCT t1.ID, t2.ID

```

```

FROM takes t1
JOIN takes t2
ON t1.course_id = t2.course_id
AND t1.sec_id = t2.sec_id
AND t1.semester = t2.semester
AND t1.year = t2.year
WHERE t1.ID < t2.ID;

-- 14
SELECT th.ID, COUNT(*)
FROM teaches th
JOIN takes t ON th.course_id = t.course_id
              AND th.sec_id = t.sec_id
              AND th.semester = t.semester
              AND th.year = t.year
GROUP BY th.ID
ORDER BY COUNT(*) DESC;

-- 15
SELECT te.ID, COUNT(t.ID)
FROM teacher te
LEFT JOIN teaches th ON te.ID = th.ID
LEFT JOIN takes t ON th.course_id = t.course_id
                  AND th.sec_id = t.sec_id
                  AND th.semester = t.semester
                  AND th.year = t.year
GROUP BY te.ID
ORDER BY COUNT(t.ID) DESC;

-- 16
SELECT th.ID, COUNT(*)
FROM teaches th
JOIN takes t ON th.course_id = t.course_id
              AND th.sec_id = t.sec_id
              AND th.semester = t.semester
              AND th.year = t.year
WHERE t.grade = 'A'
GROUP BY th.ID;

-- 17
SELECT th.ID AS enseignant, t.ID AS eleve, COUNT(*)
FROM teaches th
JOIN takes t ON th.course_id = t.course_id
              AND th.sec_id = t.sec_id
              AND th.semester = t.semester
              AND th.year = t.year
GROUP BY th.ID, t.ID;

-- 18
SELECT th.ID AS enseignant, t.ID AS eleve
FROM teaches th

```

```

JOIN takes t ON th.course_id = t.course_id
              AND th.sec_id = t.sec_id
              AND th.semester = t.semester
              AND th.year = t.year
GROUP BY th.ID, t.ID
HAVING COUNT(*) >= 2;

```

Exercice 2

1.

Relation : R(A,B,C)

- 1NF : OK
- 2NF : OK
- 3NF : OK
- BCNF : **Non** (B n'est pas super clé et $B \rightarrow C$ existe).

Décomposition en BCNF :

- R1(B, C)
- R2(A, B)

2.

Relation : R(A,B,C)

- A est superclé.
- 3NF et BCNF : **Oui**.

3.

Relation : R(A,B,C)

- 1NF : OK
- 2NF : OK
- 3NF : OK
- BCNF : **Non** ($C \rightarrow B$ mais C n'est pas super cle).

Décomposition en BCNF :

- R1(C, B)
- R2(A, C)

Exercice 3

1.

Relation : $R(A,B,C,D,E)$

Ce qu'on peut déduire avec les règles d'Armstrong :

- $A \rightarrow B$
- $A \rightarrow C$
- $A \rightarrow D$ (par transitivité : $A \rightarrow B$ et $B \rightarrow D$)
- $A \rightarrow E$ (par transitivité : $A \rightarrow B \rightarrow D$ et C donné, donc $A \rightarrow C, D \rightarrow E$)
- $A \rightarrow A$ (réflexivité)
- $B \rightarrow D$
- $B \rightarrow E$ (par transitivité : $B \rightarrow D$ et $D, C \rightarrow E$)
- $B \rightarrow A$ (par transitivité : $B \rightarrow D \rightarrow A$)
- $C, D \rightarrow E$
- $C, D \rightarrow A$ (par transitivité : $C, D \rightarrow E \rightarrow A$)
- $C, D \rightarrow B$ (par transitivité : $C, D \rightarrow E \rightarrow A \rightarrow B$)
- $C, D \rightarrow C$ (réflexivité)
- $D \rightarrow A$ (par $D \rightarrow A$ à travers E)
- $E \rightarrow A$
- $E \rightarrow B$ ($E \rightarrow A \rightarrow B$)
- $E \rightarrow D$ ($E \rightarrow A \rightarrow B \rightarrow D$)

2.

Relation : $R(A,B,C,D,E,F)$

- Fermeture de B :
 - $B \rightarrow D$
 - $D \rightarrow A$
 - $A \rightarrow B, C, D$
 - $B^+ = \{A, B, C, D\}$

B^+ tous les attributs (il manque E, F).

- Fermeture de $\{A, B\}$:

- $A \rightarrow B, C, D$
- $B, C \rightarrow D, E$
- $B \rightarrow D$
- $D \rightarrow A$

$\{A, B\}^+ = \{A, B, C, D, E\}$ (manque F).

- Montrer que $\{A, F\}$ est super-clé :

- $A \rightarrow B, C, D$
- $B, C \rightarrow D, E$
- Donc A seul + F couvre tout (si $F \rightarrow F$, ce qui est trivial).

Du coup $\{A, F\}$ est une super-clé.

- Vérification BCNF :

- Il existe des dépendances où l'antécédent n'est pas superclé .
- Du coup pas en BCNF.

3.

- Décomposition 1 : $R_1(A, B, C)$ et $R_2(A, D, E)$
- Décomposition 2 : $R_1(A, B, C)$ et $R_2(C, D, E)$

Exercice 4

```
# 1
def afficher_dependances(F):
    for dep in F:
        left, right = dep
        print(left, " --> ", right)

# 2
def afficherRelations(T):
    for rel in T:
        print(rel)

# 3
def ensembleSousParties(entree):
    result = [[]]
    for elem in entree:
        nouveaux = []
        for subset in result:
            nouveaux.append(subset + [elem])
        result.extend(nouveaux)
    sous_ensembles = []
    for s in result:
```

```

        if s:
            sous_ensembles.append(set(s))
    return sous_ensembles

# 4
def calculFermeture(attributs, F):
    fermeture = set(attributs)
    modif = True
    while modif:
        modif = False
        for gauche, droite in F:
            if set(gauche).issubset(fermeture) and not
               ↪ set(droite).issubset(fermeture):
                fermeture.update(droite)
                modif = True
    return fermeture

# 5
def fermetureComplete(F):
    result = []
    attr = set()
    for g, d in F:
        attr.update(g)
        attr.update(d)
    toutes = ensembleSousParties(attr)
    for X in toutes:
        ferm = calculFermeture(X, F)
        for Y in toutes:
            if set(Y).issubset(ferm):
                result.append((X, Y))
    return result

# 6
def implique(F, alpha, beta):
    return set(beta).issubset(calculFermeture(alpha, F))

# 7
def estSuperCle(F, R, K):
    return set(R).issubset(calculFermeture(K, F))

# 8
def estCleCandidate(F, R, K):
    if not estSuperCle(F, R, K):
        return False
    for a in K:
        if estSuperCle(F, R, K - {a}):
            return False
    return True

# 9
def trouverClesCandidates(F, R):

```



```

    cles = []
    for subset in ensembleSousParties(R):
        if estCleCandidate(F, R, subset):
            cles.append(subset)
    return cles

# 10
def findSuperKeys(F, R):
    supers = []
    for subset in ensembleSousParties(R):
        if estSuperCle(F, R, subset):
            supers.append(subset)
    return supers

# 11

# 12
def estBCNF(F, R):
    for gauche, droite in F:
        if not estSuperCle(F, R, gauche):
            return False
    return True

# 13
def schemaEnBCNF(T, F):
    for r in T:
        if not estBCNF(F, r):
            return False
    return True

```